La classification : prédire OUI ou NON

Passer d'un modèle qui prédit un nombre à un modèle qui tranche entre deux catégories. La régression logistique, les probabilités, la matrice de confusion, l'accuracy, la precision et le recall, appliqués à la question « ce chapitre va-t-il tomber au bac 2027 ? ».

1. Régression ou classification
2. Poser le problème ExamPredict
3. La régression logistique
4. La matrice de confusion
5. Accuracy, precision, recall
6. Baseline et classes déséquilibrées
7. Le seuil de décision
8. Résultats d'ExamPredict
9. La recette complète
10. Exercices corrigés
11. À savoir par cœur

---

1. Régression ou classification

> **Définition 1.1** *(apprentissage supervisé)*
>
> Un modèle **supervisé** apprend une règle à partir d'exemples dont on connaît déjà la réponse : des entrées **X** (les features) et une cible **y**. On distingue deux familles, selon la nature de y.

	RÉGRESSION	CLASSIFICATION
La cible y est	un **nombre** continu	une **catégorie** (0/1, spam/non-spam…)
Exemple ExamPredict	« 1,4 exercice de mécanique » (Day 2)	« mécanique : tombe (1) ou non (0) » (Day 3)
Modèle de base	`LinearRegression`	`LogisticRegression`
Mesure d'erreur	MAE (erreur moyenne)	accuracy, precision, recall

> **Remarque 1.2**
>
> Quand il n'y a que deux catégories, on parle de classification **binaire**. Par convention, la classe « intéressante » (fraude, malade, chapitre tombé) est notée **1**, ou **positive**.

2. Poser le problème ExamPredict

Méthode 2.1 *(construire X et y)*

Pour chaque année et chaque chapitre :

• **y** = 1 si le chapitre est tombé cette année, 0 sinon ;
• **X** = trois features : `avant_1` , `avant_2` , `avant_3` , qui valent 1 si le chapitre est tombé 1, 2 ou 3 ans plus tôt.

```
tombe = (pd.crosstab(df["annee"], df["famille"]) > 0).astype(int)   # table années × chapitres, en 0/1
```

Proposition 2.2 *(coupe dans le temps)*

Quand les données suivent le temps, le jeu de test doit être **postérieur** au jeu d'entraînement : on apprend sur 2013–2023, on teste sur 2024–2026. Mélanger les années au hasard ferait « voir le futur » au modèle : c'est une fuite de données.

3. La régression logistique

Définition 3.1

La régression logistique calcule d'abord un score linéaire, comme la régression linéaire :

$$z = w_1 \cdot \text{avant_1} + w_2 \cdot \text{avant_2} + w_3 \cdot \text{avant_3} + b$$

puis le transforme en probabilité avec la **fonction sigmoïde** :

$$p = \sigma(z) = 1 / (1 + e^{(-z)})$$

Les poids `w` et le biais `b` sont appris par `model.fit(X, y)` .

Proposition 3.2 *(propriétés de la sigmoïde)*

σ(z) est toujours strictement comprise entre 0 et 1 : c'est une probabilité. σ(0) = 0,5 ; σ(2) ≈ 0,88 ; σ(−2) ≈ 0,12. Plus z est grand, plus le modèle est sûr du OUI.

En clair : la sigmoïde est un « écraseur » qui transforme n'importe quel score en pourcentage de confiance.

MÉTHODE	RENVOIE	EXEMPLE
`model.predict(X)`	la classe : 0 ou 1 (seuil 0,5)	`[1, 0, 1]`
`model.predict_proba(X)[:, 1]`	la probabilité de la classe 1	`[0.76, 0.41, 0.53]`
`model.coef_` , `model.intercept_`	les poids w et le biais b appris	

`predict_proba` renvoie deux colonnes : la probabilité de 0, puis celle de 1. `[:, 1]` prend toutes les lignes de la deuxième colonne.

4. La matrice de confusion

Définition 4.1

La matrice de confusion compte les réponses du modèle sur le jeu de test, selon la vérité et selon la prédiction. Dans scikit-learn, les **lignes** sont la vérité et les **colonnes** la prédiction :

	PRÉDIT 0	PRÉDIT 1
VRAI 0	**VN** · vrai négatif ✅	**FP** · faux positif (fausse alerte) ❌
VRAI 1	**FN** · faux négatif (raté) ❌	**VP** · vrai positif ✅

5. Accuracy, precision, recall

Définition 5.1 *(les trois métriques)*

MÉTRIQUE	FORMULE	QUESTION POSÉE
Accuracy	(VP + VN) / total	Quelle part de réponses justes ?
Precision	VP / (VP + FP)	Quand il dit OUI, a-t-il raison ? (peu de fausses alertes)
Recall (rappel)	VP / (VP + FN)	Parmi les vrais OUI, combien en trouve-t-il ? (peu de ratés)
F1	2·P·R / (P + R)	Un compromis entre precision et recall

Méthode 5.2 *(quelle métrique privilégier ?)*

• Une **fausse alerte coûte cher** (classer un vrai mail en spam) → privilégier la **precision**.
• Un **raté coûte cher** (ne pas détecter une maladie, une fraude) → privilégier le **recall**.
• Pour ExamPredict, rater un chapitre qui tombe est pire que réviser un chapitre en trop → le **recall** compte davantage.

6. Baseline et classes déséquilibrées

Proposition 6.1 *(la baseline)*

Un modèle n'a de valeur que s'il bat la **baseline** : répondre toujours la classe la plus fréquente du jeu d'entraînement (`y_train.mode()[0]`).

Piège 6.2 *(l'accuracy qui ment)*

Sur 1 000 transactions dont 20 fraudes, un modèle qui répond toujours « pas fraude » obtient **98 % d'accuracy**… et un **recall de 0** : il ne détecte aucune fraude. Quand les classes sont déséquilibrées, on ne regarde jamais l'accuracy seule.

7. Le seuil de décision

Proposition 7.1 *(compromis precision / recall)*

Par défaut, `predict` répond 1 si la probabilité dépasse 0,5. **Baisser le seuil** fait dire OUI plus souvent : le recall monte, la precision baisse. **Monter le seuil** fait l'inverse.

```
proba = model.predict_proba(X_test)[:, 1]
pred_prudent = (proba >= 0.3).astype(int)     # plus de OUI : on rate
moins de chapitres
```

8. Résultats d'ExamPredict (physique-chimie)

TEST 2024–2026 (30 CAS)	PRÉDIT 0	PRÉDIT 1
VRAI 0	11	4
VRAI 1	5	10

MESURE	CALCUL	VALEUR
Accuracy	(11 + 10) / 30	0,70
Precision	10 / (10 + 4)	0,71
Recall	10 / (10 + 5)	0,67
F1	2 × 0,71 × 0,67 / (0,71 + 0,67)	0,69
Baseline (toujours 0)	15 / 30	0,50

Le modèle bat la baseline de 20 points. Pour 2027, il place la mécanique et la chimie organique en tête (76 %), car elles sont tombées les trois années précédentes.

Remarque 8.1 *(limite honnête)*

30 cas de test, c'est peu : un ou deux chapitres de plus ou de moins changent nettement les scores. Le résultat est un signal encourageant, pas une preuve. Le remède : plus d'années, plus de matières, plus de sessions (centres étrangers, jour 2).

9. La recette complète

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score,
confusion_matrix

train = data["annee"] < 2024                    # 1. coupe dans le temps
test = data["annee"] >= 2024

model = LogisticRegression()                    # 2. choisir le modèle
model.fit(X[train], y[train])                   # 3. apprendre sur le passé
pred = model.predict(X[test])                   # 4. prédire le test

print(confusion_matrix(y[test], pred))          # 5. mesurer
print(accuracy_score(y[test], pred), precision_score(y[test], pred),
recall_score(y[test], pred))

baseline = y[train].mode()[0]                   # 6. comparer au pif
print(accuracy_score(y[test], [baseline] * test.sum()))

model.fit(X, y)                                 # 7. réentraîner sur tout,
puis prédire le futur
proba_2027 = model.predict_proba(X_futur)[:, 1]
```

10. Exercices corrigés

Exercice 1 · Lire une matrice ★

Matrice `[[50, 10], [5, 35]]`. Calculer l'accuracy, la precision et le recall.

Corrigé. VN = 50, FP = 10, FN = 5, VP = 35. Accuracy = 85 / 100 = 0,85. Precision = 35 / 45 ≈ 0,78. Recall = 35 / 40 = 0,875.

Exercice 2 · Régression ou classification ? ★

a) le prix d'un appartement ; b) un mail est-il un spam ; c) la note au bac ; d) un patient est-il malade.

Corrigé. a) régression ; b) classification ; c) régression ; d) classification.

Exercice 3 · Le piège du déséquilibre ★★

Sur 200 patients, 10 sont malades. Un modèle répond toujours « sain ». Accuracy ? Recall ? Conclusion ?

Corrigé. Accuracy = 190 / 200 = 0,95. Recall = 0 / 10 = 0. Le modèle est inutile malgré ses 95 % : il ne détecte aucun malade.

Exercice 4 · Effet du seuil ★★

On passe le seuil de 0,5 à 0,3. Que deviennent, en général, la precision et le recall ? Pourquoi est-ce pertinent pour ExamPredict ?

Corrigé. Le modèle dit OUI plus souvent : le recall augmente, la precision diminue. Pour un élève, rater un chapitre qui tombe coûte plus cher que réviser un chapitre en trop : un seuil plus bas est justifié.

Exercice 5 · La sigmoïde ★★★

Un modèle appris donne z = 1,5 × avant_1 + 0,5 × avant_2 − 1. Calculer z et la probabilité pour un chapitre tombé l'an dernier mais pas il y a deux ans.

Corrigé. z = 1,5 × 1 + 0,5 × 0 − 1 = 0,5. p = 1 / (1 + e^(−0,5)) ≈ 0,62. Le modèle prédit OUI, avec 62 % de confiance.

11. À savoir par cœur

1. Régression = prédire un nombre. Classification = prédire une catégorie.
2. Données dans le temps : on entraîne sur le passé et on teste sur le futur.
3. Régression logistique : un score linéaire z, puis la sigmoïde 1 / (1 + e^(−z)), qui donne une probabilité.
4. `predict` donne 0 ou 1. `predict_proba(X)[:, 1]` donne la probabilité de 1.
5. Matrice de confusion : lignes = vérité, colonnes = prédiction. VN, FP, FN, VP.
6. Accuracy = (VP + VN) / total. Precision = VP / (VP + FP). Recall = VP / (VP + FN).

7	Fausses alertes coûteuses → precision. Ratés coûteux → recall.
8	Toujours comparer à la baseline. Avec des classes déséquilibrées, l'accuracy seule ment.
9	Baisser le seuil fait monter le recall et baisser la precision.

F1.DAY3.WEEK3 · Semaine 3 · Appliqué à ExamPredict AI (`src/classer_2027.py`).